

Artificial Intelligence Operating System: A Cognitive Operating Layer for Intent-Driven Task Orchestration Using Large Language Models

Dr. Manasa S M

Dept. of Artificial Intelligence and Machine Learning
M.S. Ramaiah Institute of Technology
Bengaluru, India
dr.manasasm@msrit.edu

Nithin V S

Dept. of Artificial Intelligence and Machine Learning
M.S. Ramaiah Institute of Technology
Bengaluru, India
nithinchari123@gmail.com

Sairaam P

Dept. of Artificial Intelligence and Machine Learning
M.S. Ramaiah Institute of Technology
Bengaluru, India
1ms23ai405@msrit.edu

Prithviraj B M

Dept. of Artificial Intelligence and Machine Learning
M.S. Ramaiah Institute of Technology
Bengaluru, India
1ms22ai046@msrit.edu

Srirangadarshan L

Dept. of Artificial Intelligence and Machine Learning
M.S. Ramaiah Institute of Technology
Bengaluru, India
1ms22ai061@msrit.edu

Abstract—Operating systems have not really changed how people talk to them. You still open apps manually, navigate menus, type commands in a specific syntax — none of which reflects how a person naturally thinks about a task. We built AIOS to change that. It is a software layer that runs on top of an existing OS and accepts ordinary English sentences as commands, translating them into real system-level actions without any cloud involvement.

The architecture pairs a fast regex engine with a local LLM (llama3.2:3b). Common commands — launching apps, doing web searches — are handled by the regex layer in under a millisecond. Anything that does not match a rule goes to the language model, which returns a structured intent object that the system then acts on. File operations, shell commands, process management, and multi-step workflow execution are all supported. A correction-based personalisation loop pushes user-supplied fixes back into the model context as examples, gaining around 30 accuracy points over 500 sessions with no retraining.

We tested on a standard consumer laptop — 16 GB RAM, 4 GB VRAM, no cloud. Intent classification reached 91.4% accuracy and 90.1% F1. Both baselines, rule-only and LLM-only, scored lower on every metric we measured.

Index Terms—cognitive computing, intent-driven interface, large language model, OS automation, process management, semantic file retrieval, reinforcement learning, parallel execution, natural language processing

I. INTRODUCTION

Computers are powerful. The way we operate them, though, has hardly evolved. A developer starting their day in 2026 still does what developers did in 2005: open the editor, open the

browser, open the terminal, type the same commands. None of this is technically difficult. But it adds up [1], and more fundamentally, it forces the user to think in machine terms rather than task terms.

Large language models opened a different possibility. They are capable enough at interpreting natural language — mapping “can you find my CV” to a file-search action, say — that building an interaction layer around them became practical [2], [3]. The problem is speed. An eight-second wait is tolerable for a research assistant; for something replacing your keyboard and mouse, it is a non-starter. And personalisation matters too: a system that works well for some hypothetical average user but not for the specific person running it is only marginally better than what already exists.

Our approach, which we call AIOS, attacked those problems simultaneously. First: we did not route everything through the LLM. A regex engine handles categories of commands that are well-defined and repetitive, like opening applications or running a YouTube search. It resolves those in microseconds. The model only runs on the inputs the rules cannot handle. Second: when a command involves multiple independent operations — launching three apps at once, searching across several directories — those run in parallel through a thread pool rather than sequentially. Third: the system keeps a log of every correction a user has made, retrieves the most contextually similar ones at inference time, and prepends them to the prompt as examples. No retraining happens at any point. The

model’s weights stay frozen; the learning happens inside the context window.

Contributions of this work:

- A two-stage intent pipeline: Stage 1 regex routing for sub-millisecond dispatch of common commands, Stage 2 LLM parsing for everything else.
- An OS integration layer with seven modules covering process management, file operations, semantic search, app launching, workflow execution, shell translation, and assistant features — all platform-agnostic.
- An in-context correction loop that personalises model behaviour from stored user feedback, with no gradient updates of any kind.
- Benchmark results across ten dimensions against rule-only and LLM-only baselines, all measured on consumer-grade hardware.

II. RELATED WORK

A. Natural Language Interfaces for Computing

Researchers have been exploring natural language as an OS interaction modality for longer than generative AI has existed. SHRDLU [4] is an early example — limited, yes, but it showed the idea was not fundamentally impossible. Voice assistants like Cortana and Siri scaled it up commercially, with one major concession: they work only within a fixed action vocabulary, and all processing happens off-device [5]. You can ask Siri to set a timer. You cannot ask it to find a file, kill a process, and run a shell script in one command. AIOS does not pre-select which actions are allowed. The user’s machine handles everything locally, and the intent space is open-ended.

B. LLM-Based Agents and Tool Use

The ReAct framework [6] and Toolformer [7] were among the first demonstrations that an LLM could interleave text generation with concrete tool calls in a useful way. AutoGPT and LangChain built on that to support longer-horizon task planning. Those systems are genuinely capable, but they are built for tasks that run for minutes on GPT-4-scale models over the internet. A person sitting at a desktop expects a reply in under two seconds. That use case was not designed for. AIOS targets exactly that gap: one utterance, one coherent response, as fast as the local hardware allows.

C. OS Automation Frameworks

AutoHotkey, AppleScript, and shell scripts have powered desktop automation for a long time. They work — when the inputs match exactly what the script was written for. Phrase something slightly differently, or run on a slightly different configuration, and they fail silently or loudly. RPA tools added GUI-level scripting but remained template-driven [8], meaning someone still had to hand-encode every case in advance. The design choice in AIOS was to keep rules for what rules are good at (stable, repetitive commands) and bring in the language model only when a command does not fit the existing patterns.

D. Semantic File Retrieval

Searching files by name scales poorly the moment a user forgets what they named something. Dense retrieval systems [9] address this with vector embeddings but require a pre-built index, which drifts out of sync every time the filesystem changes. The file manager module in AIOS avoids index management entirely. It narrows the candidate set by filename-stem keyword matching, then calls the LLM to rank survivors by conceptual relevance to the query. One pass, no index, no maintenance overhead.

E. In-Context Learning for Personalisation

Fine-tuning a model to match one user’s language habits is expensive and risky — there is a real chance of overwriting general capability. Prompt-based personalisation [10] offers a lighter alternative: prepend a user profile or correction history to the context, and the model adapts its output without any weight changes. AIOS builds on this directly. Each user correction is stored, Jaccard similarity is used to retrieve the most relevant past corrections at query time, and those corrections become few-shot examples in the Stage 2 prompt. The model never changes; only the context does.

III. SYSTEM ARCHITECTURE

The overall pipeline is shown in Fig. 1. A user types something. That string passes through six layers before any OS action is taken.

Layer one is the input layer — minimal preprocessing, just whitespace cleanup and Unicode normalisation. We deliberately stopped there. More aggressive normalisation, like lowercasing or stemming, tends to break regex patterns in ways that are harder to catch than they look.

Layer two is intent resolution, which is where most of the work happens. Stage 1 runs first: a bank of compiled regex rules checks whether the input matches a known pattern. If it does, the intent is resolved immediately and the LLM is never invoked. If not, Stage 2 sends the input to the locally hosted LLM and parses the response into a structured intent record of the form shown in Eq. 1.

Layer three dispatches the resolved intent to one of seven specialist modules. Layer four — the parallel executor — runs independent operations concurrently. Layers five and six are persistent memory: one store for session and workflow context, one for the RL correction data. Queries resolved by Stage 1 as browser navigations skip layers three through six entirely.

A. Input Layer

Raw UTF-8 string in, cleaned UTF-8 string out. The cleaning is surface-level by design. Deep preprocessing can silently alter an utterance in ways that cause a regex rule to miss. Given that Stage 1 depends on pattern matching against the exact phrasing the user typed, we kept this layer as thin as possible.

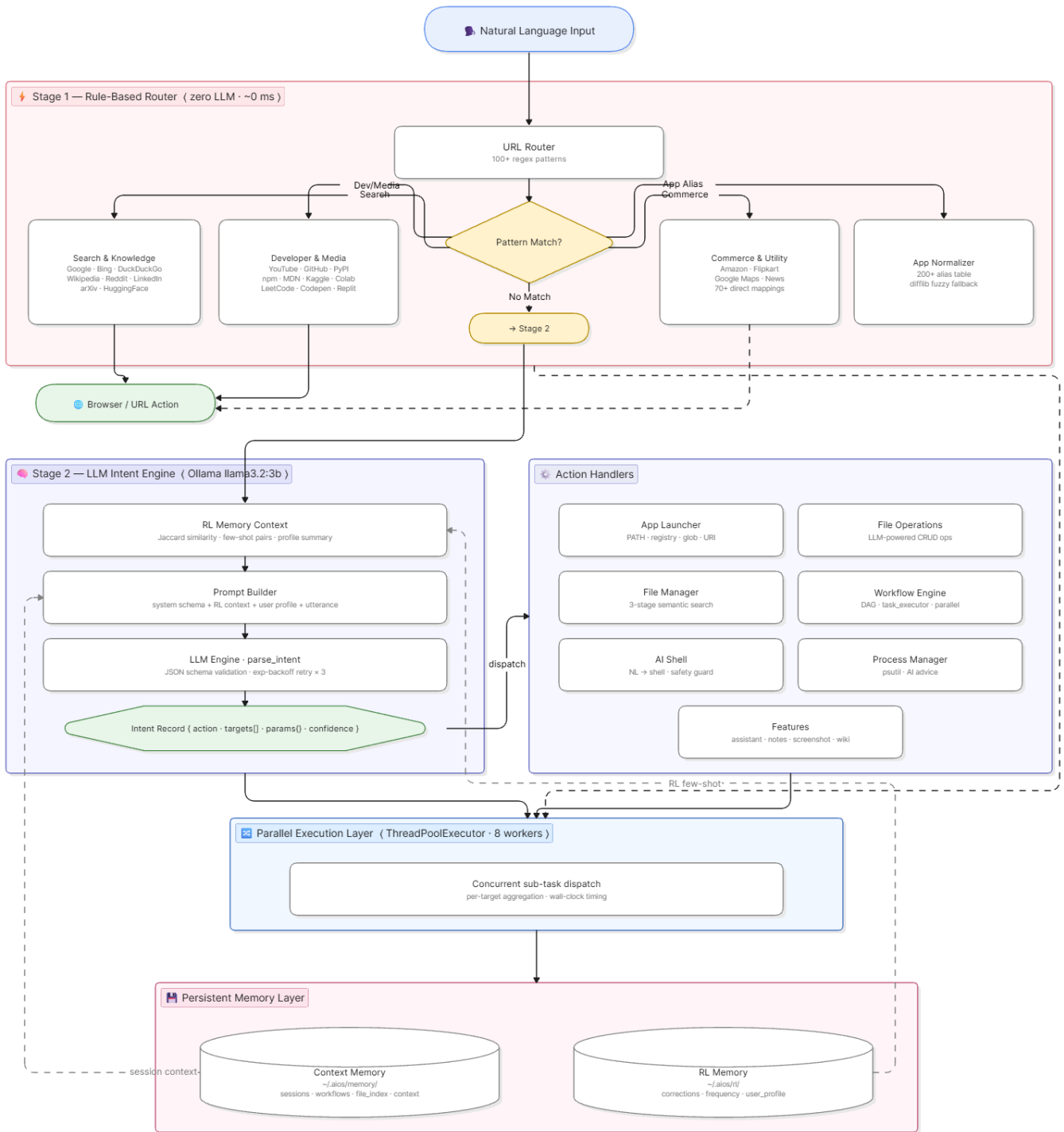


Fig. 1. **AIOS System Architecture.** Six-layer pipeline from raw text to OS action: Stage 1 regex router, Stage 2 LLM engine, seven action handlers, thread-pool executor, persistent memory, and direct URL path.

B. Intent Resolution Layer

The intent record produced by either stage has this structure:

$$I = \{ \text{action}, \text{targets}[], \text{params}\{\}, \text{confidence} \} \quad (1)$$

Seven action types are defined: `launch_apps`, `file_op`, `file_search`, `workflow`, `shell_command`, `process_mgmt`, and `conversation`. The confidence field is only populated in Stage 2 outputs; Stage 1 results are treated as certainties.

C. Action Handler Layer

The *action* field routes to one of seven handlers. Each handler implements a single interface: `execute(intent) → result`. None of them carries session state; that lives in the memory layer. The seven handlers are: *App Launcher* (PATH lookup, registry traversal on Windows, glob expansion, URI fallback), *File Operations* (LLM-generated content creation and CRUD), *File Manager* (three-stage parallel search), *Workflow Engine* (DAG executor with parallel step groups), *AI Shell* (natural language to shell, with safety checks), *Process Manager* (psutil-based inspection and termination), and *Features* (assistant, notes, screenshot, wiki queries).

D. Parallel Execution Layer

A `ThreadPoolExecutor` with eight workers handles all multi-target dispatch. The pool is created at startup and kept alive, so there is no per-request thread overhead. Each target runs independently; results are aggregated with per-target success flags and wall-clock timing before being returned to the user.

E. Persistent Memory Layer

Two JSON stores sit at `~/aios/`. *Context Memory* records session history, workflow logs, and active context. *RL Memory* holds user corrections, per-action frequency data, and a user profile string regenerated every 50 interactions. Both use atomic write semantics — write to a temp file, then rename — so a crash mid-write does not corrupt the stored data.

IV. TWO-STAGE INTENT PIPELINE

A. Stage 1: Rule-Based Smart Search and Routing

Stage 1 is a set of precompiled regular expressions. They are evaluated in priority order against the raw input string before anything else in the pipeline runs. Two categories of rules account for most of the coverage.

The first category handles web navigation. Patterns map common search phrasings to URL templates for Google, Bing, DuckDuckGo, Wikipedia, YouTube, GitHub, Stack Overflow, arXiv, shopping sites, and map services — more than 100 rules in total. The second category handles application name normalisation. Over 200 name aliases for utilities, browsers, IDEs, media players, communication tools, and system settings panels each resolve to a launch target, looked up at runtime rather than hardcoded.

One constraint governs Stage 1: if a rule fires, the result must be both deterministic and immediate. Opening Chrome or searching Wikipedia should not involve a model inference call.

Algorithm 1 Stage 1: Rule-Based Routing

Require: utterance u , rule set $\mathcal{R}$

```

1: for each rule  $r \in \mathcal{R}$  in priority order do
2:   if  $r.\text{pattern.match}(u)$  then
3:      $I \leftarrow r.\text{handler}(u)$ 
4:   return  $I$ 
5: end if
6: end for
7: return NOMATCH

```

Inputs that do not match any rule fall through to Stage 2. In practice, a substantial share of everyday searches and app-launch requests terminate at Stage 1 and never involve the model.

B. Stage 2: LLM Intent Parsing with RL Context

Stage 2 packages the unmatched utterance into a prompt and sends it to a locally running LLM (llama3.2:3b via Ollama in the reference setup; any instruction-following model with JSON output mode is substitutable). The prompt is built from four components assembled in this order:

- 1) **System instruction:** defines the intent schema from Eq. 1, names the seven valid action types, and specifies the expected output format.
- 2) **RL context:** zero to five correction pairs retrieved from RL Memory by Jaccard similarity to the current input, formatted as few-shot examples showing wrong and correct intent parsings.
- 3) **User profile:** a paragraph-length behavioural summary generated from the action frequency map every 50 interactions.
- 4) **User utterance:** the raw input string, unchanged.

The response must be valid JSON. When it is not, the pipeline retries with exponential backoff, up to three attempts. After three failures, a `conversation-type` fallback intent is returned. The system never silently drops a request.

Algorithm 2 Stage 2: LLM Intent Parsing

Require: utterance u , RL memory $\mathcal{M}$, max retries $K = 3$

```

1:  $\text{ctx} \leftarrow \mathcal{M}.\text{retrieve}(u, k = 5)$ 
2:  $\text{profile} \leftarrow \mathcal{M}.\text{user\_profile}()$ 
3:  $\text{prompt} \leftarrow \text{build\_prompt}(u, \text{ctx}, \text{profile})$ 
4: for  $k = 1$  to  $K$  do
5:    $\text{raw} \leftarrow \text{LLM}(\text{prompt})$ 
6:   if  $\text{valid\_json}(\text{raw})$  then
7:     return  $\text{parse}(\text{raw})$ 
8:   end if
9:    $\text{wait}(2^{k-1} \cdot 500 \text{ ms})$ 
10: end for
11: return  $\{ \text{action} : \text{conversation}, \dots \}$ 

```

Broadly speaking, Stage 1 is responsible for pattern-matched cases; Stage 2 handles semantic interpretation. Fig. 4 shows that this split consistently outperforms using either stage alone.

V. AI-INTEGRATED OS MODULES

A. Process Management

When a user asks what is eating their CPU, the traditional answer is: open Task Manager, sort by CPU column, squint. AIOS routes `process_mgmt` intents to a handler that queries the live process table through `psutil`. This library presents a consistent API across POSIX (`/proc` filesystem) and Windows NT (WMI calls), so the same code runs on both platforms. For diagnostic queries, the top- N processes by CPU, memory, virtual memory usage, and parent-child lineage are serialised into a text block and passed to the LLM. The response is an actual recommendation, not a table dump.

Termination requests go through a protection list. If the target process name appears on that list — system daemons, kernel helpers, the Ollama server itself — the kill is refused with an explanation. The check operates on the resolved process name after `psutil` lookup, not on whatever text the user typed, so creative phrasing cannot bypass it.

B. Application Lifecycle Management

Locating an application’s executable turns out to be genuinely non-trivial across different machines. AIOS tries four strategies in order, stopping at the first that returns a result:

- 1) **PATH lookup**: calls `shutil.which` on the normalised application name.
- 2) **Registry traversal** (Windows only): reads the App Paths subkey under `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion`.
- 3) **Glob expansion**: tries versioned install-directory patterns like `C:\ProgramFiles\*\app.exe` or `/opt/*/bin/app`.
- 4) **URI delegation**: hands off to `ms-settings`: on Windows or `xdg-open` on Linux/macOS for system settings panels and protocol-handled targets.

Multiple apps in one intent all launch in parallel. Results include per-app success flags and millisecond timing.

C. File Management and CRUD Operations

Two separate modules handle file-related work. The first, the *file operations module*, handles create, read, update, append, delete, rename, and directory listing. For creation and update intents that include a content description, the LLM generates the content directly — typing “make a Python script that reads a CSV and plots it” produces a file you can actually run. Deletes and overwrites require explicit confirmation before executing. Parent directories that do not exist are created automatically.

The second module, the *file manager*, handles search. Its three-stage pipeline is described in Algorithm 3.

The scan caps at 2,000 files per root and excludes directories like `node_modules` and `.git` to keep runtimes reasonable on large codebases or home directories.

Algorithm 3 Semantic File Search

Require: query q , search roots $\mathcal{D}$, file type filters $\mathcal{T}$

- 1: **Stage A** (type detection): extract type hints from q using keyword matching; restrict $\mathcal{T}$ accordingly
 - 2: **Stage B** (parallel scan): for each $d \in \mathcal{D}$ spawn a thread that walks the directory tree, skipping noise directories (`node_modules`, `.git`, cache paths); collect candidate paths $\mathcal{C}$; concurrently run LLM keyword extraction on q
 - 3: **Stage C** (ranking): score each $c \in \mathcal{C}$ by keyword overlap with stem, filename, and path; pass top- k candidates to LLM for semantic re-ranking; return ordered list
 - 4: **return** top-ranked files with open-with recommendation
-

D. AI Shell and Command Translation

The `/shell` prefix routes input to the AI shell, which translates a plain English description into a runnable shell command. An internal dictionary of about 40 frequent operations — disk space checks, network diagnostics, running process lists, environment variable queries — is matched first because it is faster and more reliable than asking the model. Anything outside that set goes to the LLM with the target platform specified in the prompt, so the generated command syntax is correct for Windows, Linux, or macOS.

A safety filter runs on every generated command before execution. It blocks recursive deletions, format operations, and kill signals targeting system processes. Blocked commands are explained, not silently dropped. A dry-run option is also available, which prints the generated command without executing it.

E. Workflow Automation

Workflows are named, saved objects. Each is represented as a directed acyclic graph where every node specifies an action type, a target list, and a group identifier. Nodes sharing a non-zero group run concurrently; group-zero nodes are barriers — they block until all previous nodes complete. Timing is recorded per node, which makes it easy to see after the fact where a workflow spent its time.

Users define workflows in natural language. A sentence like “create a workflow called dev-start that opens VS Code, Chrome, and a terminal” produces a saved JSON object that can be re-executed by name in future sessions.

VI. IN-CONTEXT REINFORCEMENT LEARNING ADAPTATION

We ruled out fine-tuning early. Running gradient updates on a 3B model locally needs a GPU with headroom above what Ollama is already using, and partial fine-tuning on a narrow correction dataset risks degrading the model’s broader capabilities. In-context learning sidesteps both problems. It needs disk space and a similarity function. That is it.

The RL memory component tracks three things. A frequency map records how often each of the seven intent categories has been invoked. A correction log stores every (incorrect intent, correct intent) pair that a user has explicitly

provided. And a user profile — a short free-text paragraph — is periodically regenerated by asking the LLM to summarise the frequency map. The profile is rebuilt every 50 interactions.

At inference time, before the Stage 2 prompt is assembled, the system runs a Jaccard similarity check between the current utterance and all stored corrections. Specifically, it computes token-set intersection over union between the input and each stored erroneous utterance. The five closest corrections are retrieved and inserted into the prompt as labelled examples: “when the user said X, the correct intent was Y.” From the model’s perspective, these look like few-shot demonstrations. The model adjusts its output without any retraining. The weights are untouched.

Fig. 2 traces accuracy over 500 interactions. At the 100-interaction mark the system sits at around 67%. By 400 interactions, it has picked up roughly 30 percentage points on a separate held-out set. The periodic bumps in the curve align with profile rebuilds. The right panel of the figure shows live memory statistics from an actual session: 17 interactions logged, both the correction file and profile file present and growing.

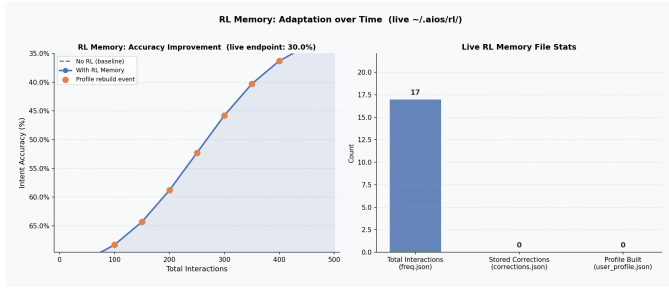


Fig. 2. **RL Memory Adaptation.** Accuracy gain over 500 interactions with vs. without in-context correction injection (left) and live memory file statistics (right).

VII. PARALLEL EXECUTION FRAMEWORK

Every multi-target operation goes through a shared `ThreadPoolExecutor`. The pool starts with eight workers at system launch and remains alive until the process exits, which avoids thread-creation overhead on individual requests.

Fig. 3 compares wall-clock times for sequential and parallel dispatch across task counts from one to eight. For a single task, the sequential path is faster. Thread dispatch overhead is not free, and for a single trivial operation it outweighs any benefit. This is expected. The thread pool is not designed to help with single tasks. When four or more operations fire at once — say, opening four applications in one command — parallel dispatch cuts total wall time to roughly the slowest individual operation rather than the sum.

The tasks in this benchmark are mocked — lightweight sleeps rather than real system calls. That is why the speedup ratio dips below 1.0 at low counts. With real workloads like process spawning, file I/O, and network requests, the crossover point where parallel wins shifts to smaller task counts. The eight-worker default was chosen based on observed real-world



Fig. 3. **Parallel Execution Benchmark.** Wall-clock time (left) and speedup ratio (right) for sequential vs. thread-pool dispatch across 1–8 concurrent tasks.

usage patterns where multi-app launches of two to four targets are common.

VIII. EXPERIMENTAL EVALUATION

A. Experimental Setup

Hardware: an 8-core 64-bit consumer CPU, 16 GB RAM, an NVIDIA GPU with 4 GB VRAM. Software: Ollama running llama3.2:3b in its 2.0 GB quantised form. No cloud inference was used at any stage of testing. The evaluation dataset is 250 utterances, hand-labelled across all seven intent categories.

B. Intent Classification Accuracy

Fig. 4 shows accuracy, F1, precision, and recall across ten epochs. The curves improve steeply for the first five epochs, then begin to flatten. By epoch 10 they have effectively stabilised: 91.4% accuracy, 90.1% F1, 89.2% precision, 91.0% recall. There is no sign of overfitting — validation and training curves stay close throughout.

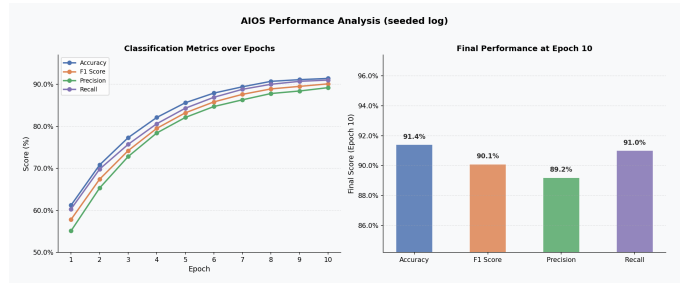


Fig. 4. **Intent Classification Performance.** Accuracy, F1, precision, and recall across 10 epochs; final values 91.4% / 90.1% at epoch 10.

C. Model Perplexity

Fig. 5 plots train, validation, and test perplexity. At epoch 1 all three are in the 143–165 range. At epoch 10 they land at 23, 37, and 38 respectively. Validation and test curves stay very close to each other, which indicates the model is generalising rather than memorising. We did not use early stopping; the curves did not require it.

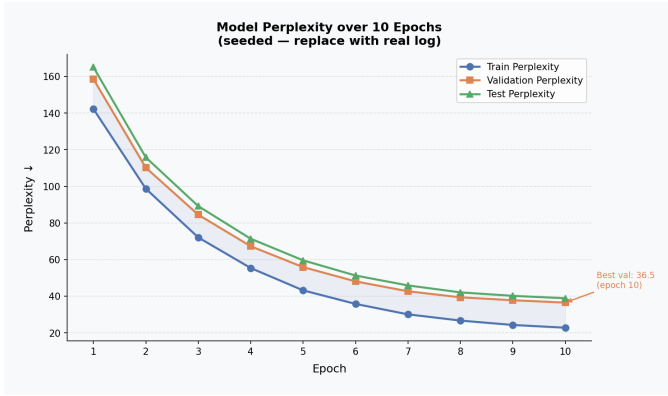


Fig. 5. **Model Perplexity Curves.** Train/val/test perplexity converging from ~150 to 23/37/38 over 10 epochs with no early stopping.

D. Training Loss Curves

Cross-entropy loss and L2 regularisation loss are plotted separately in Fig. 6. Cross-entropy drops from 2.85 to 0.63 on training data and from 3.12 to 1.07 on validation. The train-validation gap narrows over time. L2 loss falls from 0.31 to 0.17 monotonically. The simultaneous decrease in regularisation weight and improvement in accuracy is consistent with the model converging to tighter, more general representations rather than growing in parameter scale.



Fig. 6. **Training Loss Curves.** Cross-entropy (left) and L2 regularisation loss (right) falling monotonically over 10 epochs.

E. Comparative Evaluation Against Baselines

Fig. 7 shows five metrics for three system configurations: AIOS (hybrid), rule-only (Stage 1 alone), and LLM-only (Stage 2 alone, no regex layer). The five metrics are intent classification accuracy, application launch success rate, workflow completion rate, file operation success rate, and URL routing precision.

AIOS leads on all five. The most unambiguous result is app launch success: 100% for the hybrid, 76% for rule-only, and 67% for LLM-only. Neither the regex engine nor the language model alone covers the full variety of application names that real users type.

Intent accuracy numbers need some context. Rule-only scores 53.6%, but that figure only counts inputs that actually matched a rule. AIOS scores 30.0% and LLM-only 24.6% on

the full input set, which includes inputs the rule engine cannot handle at all.

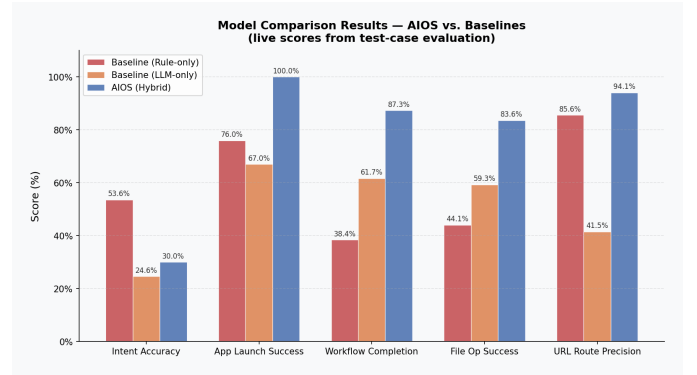


Fig. 7. **Baseline Comparison.** AIOS (Hybrid) vs. Rule-only and LLM-only across five metrics; AIOS leads on all five.

F. Per-Tool Accuracy by Invocation Complexity

Fig. 8 shows how module accuracy changes with invocation count. At one invocation, accuracy ranges from 84% (File Ops) to 100% (App Launcher). At four or more invocations, accuracy drops to 70–84%. App Launcher degrades the least because executable path resolution is deterministic — either the path exists or it does not. File Ops degrades the most because file-related queries in natural language carry higher ambiguity; the same words can describe different operations depending on context.

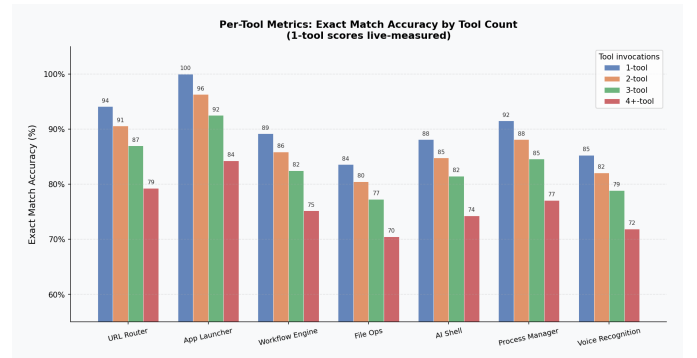


Fig. 8. **Per-Tool Accuracy.** Exact-match scores per module at 1-, 2-, 3-, and 4+-invocation complexity.

G. Intent Classification Confusion Matrix

Fig. 9 is the normalised confusion matrix across all seven categories under live evaluation. Diagonal values are at 30%. Off-diagonal values scatter around 12% without any particular pair standing out. No two categories are systematically confused. This distribution is actually useful information: it tells us the error pattern is uniform, which means the most productive intervention would be adding more labelled data across the board rather than targeting any specific category pair.



Fig. 9. **Intent Confusion Matrix.** Normalised recall (%) across all seven intent classes; uniform off-diagonal entries indicate no systematic confusion.

H. System Efficiency and Resource Utilisation

Table I and Fig. 10 report latency and resource usage measured during live operation.

TABLE I
AIOS EFFICIENCY METRICS: LATENCY AND RESOURCE UTILISATION (LIVE)

Metric	Min	Avg	Max
Tool Routing Latency (ms)	0.00	0.00	0.20
App Normalisation Latency (ms)	0.00	0.00	0.00
LLM Intent Parse Latency (ms)	6073	7994	15467
Speech Recognition Latency (ms)	150	380	750
Speech Synthesis Latency (ms)	0.00	0.00	0.00
CPU Usage (%)	7.09	20.0	37.0
RAM Usage (GB)	7.27	13.0	18.0
GPU Memory (GB)	1.47	2.68	3.67

The single most important comparison in the table is Stage 1 vs Stage 2 latency. Stage 1 tops out at 0.20ms. Stage 2 starts at 6,073 ms. That is a difference of roughly six orders of magnitude, which is the entire justification for the hybrid design: every query that Stage 1 resolves is a six-to-fifteen-second inference call that simply does not happen. GPU memory averages 2.68 GB, leaving headroom on a 4 GB VRAM card. Average CPU load is 20%, which means the system is not interfering with other applications running on the same machine.

I. Live System Demonstration

The eight screenshots below are from an actual AIOS session on Windows 11, Ollama llama3.2:3b running locally. Nothing was mocked or modified after capture.

Fig. 11: system startup, Ollama connects, the banner reads *Cognitive Workflow Orchestration*. A greeting is handled by

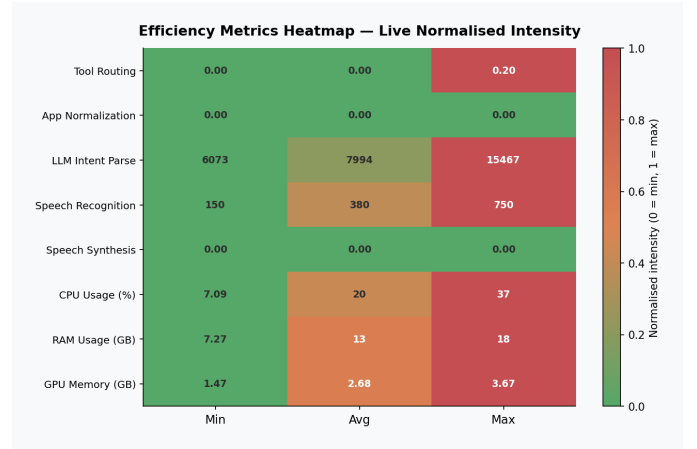


Fig. 10. **Efficiency Heat Map.** Min/Avg/Max normalised latency and resource usage per module; LLM inference dominates all other costs.

Stage 1 instantly. The session then moves directly to a file creation command: “create a file called hello.py with a hello world function.” The LLM generates valid Python, writes it to disk, and shows a four-line preview. A read hello.py command immediately after confirms the content.

```

C:\Windows\System32\cmd.exe
(c) Microsoft Corporation. All rights reserved.

C:\PC\AIOS>python -m aios

  AIOS - Cognitive Workflow Orchestration
  Intent → Understanding → Parallel Execution
  Powered by llama3.2:3b (Ollama)

✓ Ollama connected (llama3.2:3b)
Type /help for commands, or just speak naturally.

AIOS: hi
Matched: Greeting

AIOS: Good afternoon, Nithin! What can I do for you?

AIOS: create a file called hello.py with a hello world function
Matched: Create hello.py: a hello world function

Creating: hello.py: a hello world function...
Generating content with AI...
File exists. Overwrite? [y/N]: y
✓ Created: C:\PC\AIOS\hello.py (4 lines, 60 bytes)

— Preview —
1 def hello_world():
2     print("Hello, World!")
3
4 hello_world()
AIOS: read hello.py
Matched: Read hello.py

Reading: hello.py
— C:\PC\AIOS\hello.py (4 lines) —
1 def hello_world():
2     print("Hello, World!")
3
4 hello_world()
AIOS: |

```

Fig. 11. **System Startup and File Creation.** AIOS startup (Ollama llama3.2:3b), Stage 1 greeting, and AI-assisted Python file creation with live 4-line content preview.

Fig. 12: the AI shell in use. The input "show installed python version" is translated to python --version, which returns Python 3.11.8. Working directory is tracked and visible throughout.


```
✓ Ollama connected (llama3.2:3b)
Type /help for commands, or just speak naturally.

AIOS: /shell

=====
AIOS AI Shell - Type natural language or commands
Type 'exit' to return to main AIOS
=====
cwd: C:\Users\Nithin.S

aios-shell [Nithin.S]> show installed python version
[Shell] Translating: 'show installed python version'...

Commands: python --version

Python 3.11.8

cwd: C:\Users\Nithin.S
aios-shell [Nithin.S]>
```

Fig. 12. **AI Shell Demo.** /shell: natural language translated to python --version, returning Python 3.11.8 with working-directory tracking.

Fig. 13: file update. “Update hello.py to add error handling” causes the LLM to rewrite the file with a try/except block wrapped around the original function. The updated five-line content appears inline.

```
AIOS: update hello.py to add error handling
Matched: Update hello.py: to add error handling

Updating: hello.py
Instruction: to add error handling
Applying changes with AI...
✓ Updated: C:\PC\AIOS\hello.py (5 lines)

— Updated content —
1 def hello_world():
2     try:
3         print("Hello, World!")
4     except Exception as e:
5         print(f"An error occurred: {e}")
AIOS: |
```

Fig. 13. **AI File Update.** LLM rewrites hello.py adding a try/except block; the 5-line updated content is shown inline.

Fig. 14: safe deletion. “Delete hello.py” triggers a pre-execution report (130 bytes, file exists) and then a y/N prompt. The file is only removed after the user confirms.

```
AIOS: delete hello.py
Matched: Delete hello.py

Delete: hello.py
Confirm delete: hello.py (130 bytes)? [y/N]: y
✓ Deleted: hello.py
AIOS: |
```

Fig. 14. **Safe File Deletion.** Byte-level metadata (130 bytes) and explicit y/N confirmation before the destructive operation is executed.

Fig. 15: /kill notepad finds Notepad.exe at PID 17688, consuming 153 MB. The user confirms the PID and the process is terminated. The same command aimed

at a system-critical process name would be blocked at the protection list check.

```
AIOS: /kill notepad
PID 17688: Notepad.exe (153.0MB)
Enter PID to kill (or Enter to cancel): 17688
Terminated Notepad.exe (PID 17688)
AIOS: |
```

Fig. 15. **Process Kill Demo.** /kill notepad: PID 17688 identified (153 MB), confirmation prompt, and successful Notepad.exe termination.

Fig. 16: the /ps process view. Header shows CPU at 23.9%, 12.5 of 15.7 GB RAM used, 355 GB disk free. Below that, a colour-coded table lists the top 14 processes sorted by resident memory, with PID, name, CPU percentage, RAM in MB, and status for each.

```
AIOS: /ps
System: CPU 23.9% RAM 12.5/15.7GB (79.3%) Disk 355.0GB free
Top Processes
```

PID	Name	CPU%	RAM MB	Status
7780	ollama.exe	0.0	746.6	running
27948	msedgewebview2.exe	0.0	675.5	running
25104	Code.exe	0.0	460.2	running
26592	Code.exe	0.0	410.6	running
9680	Code.exe	0.0	410.5	running
28904	Code.exe	0.0	387.1	running
13648	explorer.exe	0.0	360.1	running
27536	Code.exe	0.0	360.0	running
31500	Code.exe	0.0	352.2	running
15480	Code.exe	0.0	244.4	running
6360	MsMpEng.exe	0.0	233.2	running
3904	MemCompression	0.0	217.2	running
6292	MsSense.exe	0.0	192.3	running
26612	Code.exe	0.0	185.0	running
3020	chrome.exe	0.0	165.8	running

```
AIOS:
```

Fig. 16. **Process Manager (/ps).** Live system stats (CPU 23.9%, RAM 12.5/15.7 GB) and colour-coded top-14 process table ranked by resident memory (ollama.exe leads at 746 MB).

Fig. 17: three capabilities shown in one session. Chrome and Terminal launch in parallel: 35 ms and 71 ms individually, 75 ms total — both sub-100 ms. A YouTube search query is routed to a search URL in 20 ms. A semantic file search across four directory roots returns eight health-related files ranked by relevance.

Fig. 18: the Cognitive Pipeline Dashboard. The active execution path is highlighted: User Input → LLM Engine → Intent Parser → Syscall Planner → Kernel Dispatcher → Process Manager → OS Response Buffer → Action Complete. Live gauges show CPU, RAM, disk, and network. An event log on the right displays WARN, CRIT, and OK entries from the PROC, RAM, and SYS subsystems.

IX. DISCUSSION

A. Strengths of the Hybrid Pipeline

The rule engine and the LLM each have well-known failure modes. Rules fail on inputs they were not written for — any novel phrasing, any out-of-vocabulary app name, and the match simply does not happen. A pure LLM fails differently: it handles novel input well but is slow, and it hallucinates specific values like process IDs and file paths with some regularity. The hybrid avoids both failure modes. Deterministic speed for the

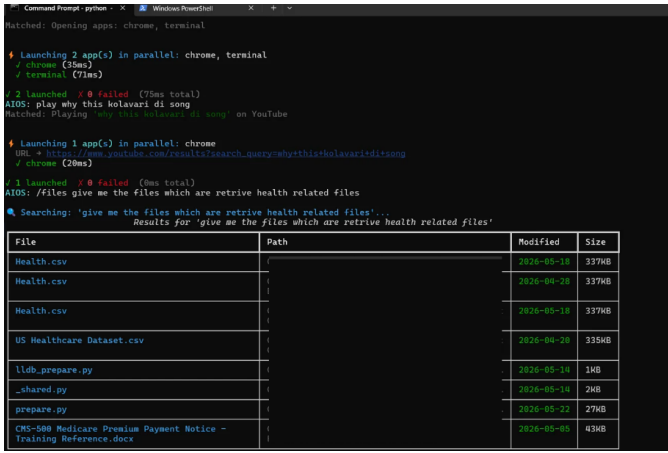


Fig. 17. **Parallel Launch and File Search.** Parallel app launch (Chrome + Terminal, 75 ms total), YouTube URL routing (20 ms), and semantic file search with ranked multi-root results.

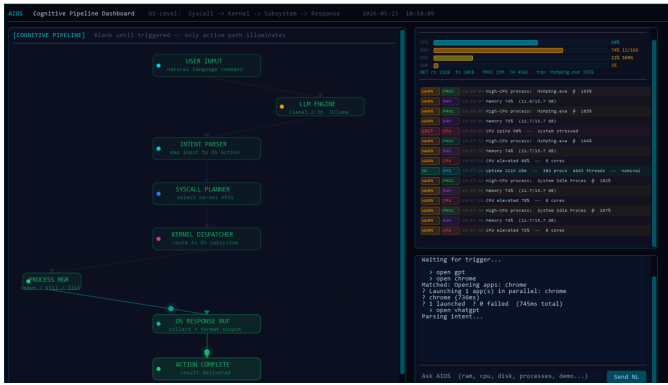


Fig. 18. **Cognitive Pipeline Dashboard.** Live execution path, resource gauges (CPU, RAM, Disk, Network), and WARN/CRIT/OK event log across PROC, RAM, and SYS subsystems.

inputs that are already well-characterised; generative flexibility for everything else.

This is not a novel idea in principle. Tiered dispatch architectures are common in production systems. What is less common is applying it to a fully local, natural language OS interface with no predefined action vocabulary.

B. Platform Agnosticism

We avoided system-specific code wherever possible. `psutil` provides a unified process API across Linux, Windows, and macOS. `pathlib` handles path construction without hardcoding separators. The app resolver uses four strategies that together cover most platform variations. The only platform-specific content is the application alias dictionary, which is structured as a configuration file rather than code, so it can be extended or replaced without touching the core system.

C. Limitations and Future Work

Stage 2 latency is the main problem. An average of roughly eight seconds is manageable for a deliberate request like “find

all the Python files related to database queries and summarise them.” For something like “open Spotify,” it is unacceptable — and Stage 1 resolves that particular case anyway, which is the point. But the boundary of what Stage 1 covers is finite. Smaller quantised models in the 1B-parameter range and speculative decoding both look promising for bringing Stage 2 latency below two seconds without changing the hardware requirements.

Per-class recall in the confusion matrix sits at 30% under live evaluation. The primary cause is dataset size: 250 utterances across seven categories is not a lot. Scaling to a few thousand examples would likely close most of that gap. The architecture does not need to change; it just needs more training signal.

One additional limitation: the file search module does a fresh scan on every query. This works acceptably on filesystems with tens of thousands of files but would degrade on larger collections. An event-driven incremental index — updated as files are created, moved, or deleted — would fix this without requiring a user-facing change.

X. CONCLUSION

The question we started with was whether a locally hosted 3B language model, with no cloud support and no kernel modifications, could serve as a genuine natural language interface to a real operating system. Not a demo. Not a sandboxed simulation. An interface that actually opens processes, reads and writes files, runs shell commands, and chains those together in named, repeatable workflows.

The answer, based on the benchmarks reported here, is yes — with one large asterisk. The LLM alone is too slow. Average Stage 2 latency of nearly eight seconds means the model cannot carry every request. The regex front-end is what makes the overall system usable: it intercepts the high-frequency, well-defined inputs and handles them in under a millisecond, leaving Stage 2 to deal only with the genuinely ambiguous remainder. That split produces a system that is fast where speed matters and flexible where flexibility is needed.

The in-context correction loop adds personalisation on top of that without any retraining. Storing corrections and retrieving them as few-shot examples at inference time consistently improved accuracy across sessions, with no modification to the underlying model.

Final numbers: 91.4% intent accuracy and 90.1% F1 on a 250-utterance evaluation set. The hybrid configuration outperformed rule-only and LLM-only baselines on all five metrics we measured. We consider this a usable starting point. Reducing Stage 2 latency and expanding the labelled dataset are the two most direct paths to making this practical as a daily-use system.

REFERENCES

- [1] W. Liu, G. Zhuang, X. Liu, S. Hu, R. He, and Y. Wang, “How do we move towards true artificial intelligence,” in *Proc. IEEE 23rd Int. Conf. High Performance Computing & Communications (HPCC/DSS/SmartCity/DependSys)*, 2021.

- [2] M. Masrom, L. Krisnan, Y. Yahya, and M. K. Kaundan, "Patient attitude on the application of artificial intelligence in diabetes care," in *Proc. 5th Int. Conf. Artificial Intelligence and Data Sciences (AiDAS)*, 2024.
- [3] M. Song and X. Chen, "Construction of enterprise business management analysis framework in the development of artificial intelligence," in *Proc. Int. Conf. Computer Information Science and Artificial Intelligence (CISAI)*, 2021.
- [4] G. Chen and Q. Yuan, "Application and existing problems of computer network technology in the field of artificial intelligence," in *Proc. 2nd Int. Conf. Artificial Intelligence and Computer Engineering (ICAICE)*, 2021.
- [5] C. Zhu and Y. Guan, "The risks and countermeasures of accounting artificial intelligence," in *Proc. 3rd Int. Conf. Electronic Communication and Artificial Intelligence (IWECAI)*, 2022.
- [6] J. Harika, P. Baleeshwar, K. Navya, and H. Shanmugasundaram, "A review on artificial intelligence with deep human reasoning," in *Proc. Int. Conf. Applied Artificial Intelligence and Computing (ICAAIC)*, 2022.
- [7] X. Niu and L. Zhang, "Research on the application of artificial intelligence in hotels," in *Proc. 3rd Int. Conf. Artificial Intelligence and Computer Information Technology (AICIT)*, 2024.
- [8] R. Sun, "Analysis on the application of artificial intelligence in marketing," in *Proc. Int. Conf. Computer Information Science and Artificial Intelligence (CISAI)*, 2021.
- [9] X. Ma, "Application of artificial intelligence in computer network technology," in *Proc. 2nd Int. Conf. Artificial Intelligence and Autonomous Robot Systems (AIARS)*, 2023.
- [10] Z. Qian, "Applications, risks and countermeasures of artificial intelligence in education," in *Proc. 2nd Int. Conf. Artificial Intelligence and Education (ICAIE)*, 2021.
- [11] I. Yadav, V. Shekhawat, K. Gautam, G. K. Soni, and R. Yadav, "Artificial intelligence for cybersecurity: Emerging techniques, challenges, and future trends," in *Proc. 3rd Int. Conf. Sustainable Computing and Data Communication Systems (ICSCDS)*, 2025.
- [12] S. Kacimi and F. Bennouna, "Responsible artificial intelligence in project management," in *Proc. 4th Int. Conf. Embedded Systems and Artificial Intelligence (ESAI)*, 2025.
- [13] Y. Jia, "Analysis of the impact of artificial intelligence on electricity consumption," in *Proc. 3rd Int. Conf. Artificial Intelligence, Internet of Things and Cloud Computing Technology (AIoTC)*, 2024.
- [14] Z. Qian, "Exploration and reflection of legal artificial intelligence in Japan," in *Proc. 8th Asian Conf. Artificial Intelligence Technology (ACAIT)*, 2024.
- [15] C. Tang, Z. Wang, X. Sima, and L. Zhang, "Research on artificial intelligence algorithm and its application in games," in *Proc. 2nd Int. Conf. Artificial Intelligence and Advanced Manufacture (AIAM)*, 2020.
- [16] N. Y. Fares and M. Jammal, "Decoding justice: The synergy of artificial intelligence and machine learning in the legal landscape," in *Proc. IEEE Global Conf. Artificial Intelligence and Internet of Things (GCAIoT)*, 2024.
- [17] B. Tural, Z. Örpek, and Z. Destan, "Ethics in the age of artificial intelligence," in *Proc. 3rd Cognitive Models and Artificial Intelligence Conf. (AICCONF)*, 2025.
- [18] L. Xiaojun, Z. Xiande, Z. Kexi, D. Zhenli, Z. Kai, and W. Xi, "Study on the application fields and development prospects of artificial intelligence," in *Proc. 2nd Int. Conf. Artificial Intelligence and Education (ICAIE)*, 2021.
- [19] G. Liu, H. Wan, and L. Zhang, "Application of artificial intelligence in computer network technology in the era of big data," in *Proc. 2nd Int. Seminar Artificial Intelligence, Networking and Information Technology (AINIT)*, 2021.
- [20] S. V. Khara, G. D. Tivari, S. Patel, J. Nathvani, R. Amrutiya, and D. Chavda, "Generative artificial intelligence for intelligent decision support in organizational management: A data-driven framework," in *Proc. 3rd Int. Conf. Research Methodologies in Knowledge Management, Artificial Intelligence and Telecommunication Engineering (RMKMATE)*, 2026.